

A Scalable Framework for Wireless Distributed Computing

Songze Li, *Student Member, IEEE*, Qian Yu, Mohammad Ali Maddah-Ali, *Member, IEEE*,
and A. Salman Avestimehr, *Senior Member, IEEE*

Abstract—We consider a wireless distributed computing system, in which multiple mobile users, connected wirelessly through an access point, collaborate to perform a computation task. In particular, users communicate with each other via the access point to exchange their locally computed intermediate computation results, which is known as *data shuffling*. We propose a scalable framework for this system, in which the required communication bandwidth for data shuffling does not increase with the number of users in the network. The key idea is to utilize a particular repetitive pattern of placing the data set (thus a particular repetitive pattern of intermediate computations), in order to provide the coding opportunities at both the users and the access point, which reduce the required uplink communication bandwidth from users to the access point and the downlink communication bandwidth from access point to users by factors that grow linearly with the number of users. We also demonstrate that the proposed data set placement and coded shuffling schemes are optimal (i.e., achieve the minimum required shuffling load) for both a centralized setting and a decentralized setting, by developing tight information-theoretic lower bounds.

Index Terms—Wireless distributed computing, edge computing, coding, information theory, scalability.

I. INTRODUCTION

RECENT years have witnessed a rapid growth of computationally intensive applications on mobile devices, such as mapping services, voice/image recognition, and augmented reality. The current trend for developing these applications is to offload computationally heavy tasks to a “cloud”, which has greater computational resources. While this trend has its merits, there is also a critical need for enabling *wireless distributed computing*, in which computation is carried out using the computational resources of a cluster of wireless devices collaboratively. Wireless distributed computing eliminates, or at least de-emphasizes, the need for a core computing environment (i.e., the cloud), which is critical in several important applications, such as autonomous control and navigation

for vehicles and drones, in which access to the cloud can be very limited. Also as a special case of the emerging “Fog computing architecture” [3], it is expected to provide significant advantages to users by improving the response latency, increasing their computing capabilities, and enabling complex applications in machine learning, data analytics, and autonomous operation (see e.g., [4]–[6]).

Without the help from a centralized cloud, the local computing capability of a wireless device is often limited by its local storage size. For example, for a mobile navigation application in which a smart car wants to compute the fastest route to its destination, over a huge dataset containing the map information and the traffic conditions over a period of time, the local storage size of an individual car is too small to store the entire dataset, and hence individual processing is not feasible. However, using a wireless distributed computing framework, a group of smart cars, each storing a part of the dataset, can collaborate to meet their respective computational needs over the entire dataset.

The major challenge in developing a scalable framework for wireless distributed computing is the significant communication load, required to exchange the intermediate computation results among the mobile users. In fact, even when the processing nodes are connected via high-bandwidth inter-server communication bus links (e.g., a Facebook’s Hadoop cluster), it is observed in [7] that 33% of the job execution time is spent on data shuffling. The communication bottleneck is expected to get much more severe as we move to a wireless medium where the communication resources are much more scarce. More generally, as the network size increases, while the computation resources grow linearly with network size, the overall communication bandwidth is fixed and can become the bottleneck. This raises the following fundamental question.

Is there a *scalable* framework for wireless distributed computing, in which the required communication load is fixed and independent of the number of users?

Our main contribution is to provide an affirmative answer to this question by developing a framework for wireless distributed computing that utilizes redundant computations at the users, in order to create coding opportunities that reduce the required communication, achieving a scalable design. The developed framework can be considered as an extension of our previously proposed *coded distributed computing* framework for a wireline setting in [8]–[11], into the wireless distributed computing domain. To develop such a framework, we exploit

Manuscript received August 27, 2016; revised March 23, 2017; accepted April 26, 2017; approved by IEEE/ACM TRANSACTIONS ON NETWORKING Editor D. Leith. Date of publication May 29, 2017; date of current version October 13, 2017. This work was supported in part by the NSF under Grant CCF-1408639 and Grant NETS-1419632, in part by the ONR under Award N000141612189, in part by the National Security Agency (NSA) under Award H98230-16-C-0255, in part by Intel under a research gift, and in part by the Defense Advanced Research Projects Agency under Grant HR001117C0053. A part of this paper was presented in IEEE/ACM SEC 2016 [1]. A shorter version of this paper was presented in IEEE GLOBECOM 2016 [2]. (Corresponding author: Songze Li.)

S. Li, Q. Yu, and A. S. Avestimehr are with the Department of Electrical Engineering, University of Southern California, Los Angeles, CA 90089 USA (e-mail: songzeli@usc.edu; qyu880@usc.edu; avestimehr@ee.usc.edu).

M. A. Maddah-Ali is with the Department of Electrical Engineering, Sharif University of Technology, Tehran 11365, Iran (e-mail: maddah_ali@sharif.edu).

Digital Object Identifier 10.1109/TNET.2017.2702605

three opportunities in conjunction:

- 1) **Side-Information:** When a sub-task has been processed in more than one node, the resulting intermediate outcomes will be available in all those nodes as side-information. This provides some opportunities for *coding* across the results and creates packets that are useful for multiple nodes.
- 2) **Coding:** We use coding to develop packets useful to more than one mobile users. This allows us to exploit the *multicasting* environment of the wireless medium and save communication overhead.
- 3) **Multicasting:** Wireless medium by nature is a multicasting environment. It means that when a signal is transmitted, it can be heard by all the nodes. We exploit this phenomenon by creating and transmitting signals that help several user nodes simultaneously.

A. Motivating Example

Let's first illustrate our scalable design of wireless distributed computing through an example. Consider a scenario where 3 mobile users want to run an application (e.g., image recognition). Each user has an input (e.g., an image) to process using a dataset (e.g., a feature repository of objects) provided by the application. However, the local memory of an individual user is too small to store the entire dataset, and they have to collaboratively perform the computation. The entire dataset consists of 6 equally-sized files, and each user can store at most 4 of them. The computation is performed distributedly following a commonly used MapReduce-like distributed computing structure (see e.g., MapReduce [12] and Spark [13]). More specifically, every user computes a Map function, for each of the 3 inputs and each of the 4 files stored locally, generating 12 intermediate values. Then the users communicate with each other via an access point they all wirelessly connect to, which we call *data shuffling*. After the data shuffling, each user knows the intermediate values of his own input in all 6 files, and passes them to a Reduce function to calculate the final output result.

During data shuffling, since each user already has 4 out of 6 intended intermediate values locally, she would need the remaining 2 from the other users. Thus, one would expect a communication load of 6 (in number of intermediate values) on the uplink from users to the access point and 6 on the downlink in which the access point simply forwards the intermediate values to the intended users. However, we can take advantage of the opportunities mentioned above to significantly reduce the communication loads. As illustrated in Fig. 1, through careful placement of the dataset into users' memories, we can design a coded communication scheme in which every user sends a bit-wise XOR, denoted by $\oplus$, of 2 intermediate values on the uplink, and then the access point, without decoding any individual value, simply generates 2 random linear combinations $C_1(\cdot, \cdot, \cdot)$ and $C_2(\cdot, \cdot, \cdot)$ of the received messages and broadcasts them to the users, simultaneously satisfying all data requests. Using this coded approach, we achieve an uplink communication load of 3 and a downlink communication load of 2.

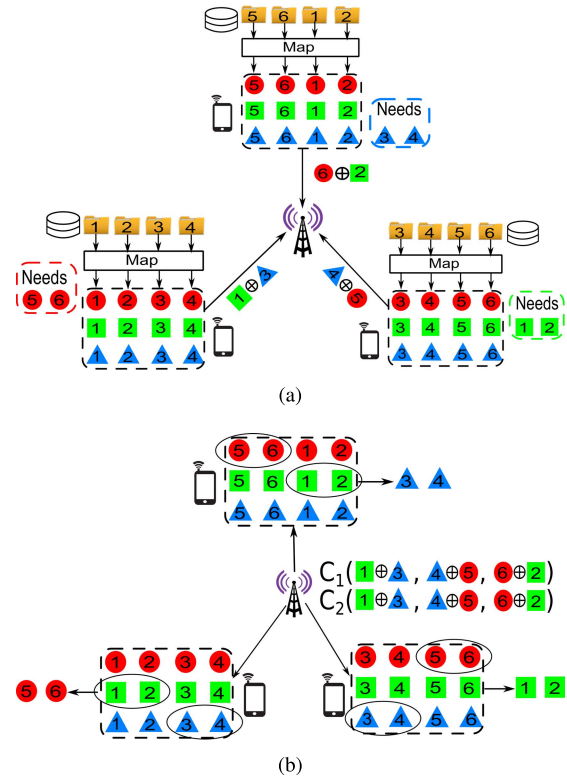


Fig. 1. Illustration of the CWDC scheme for an example of 3 mobile users. (a) Uplink. (b) Downlink.

We generalize the above example by designing a coded wireless distributed computing (CWDC) framework that applies to arbitrary type of applications, network size and storage size. In particular, we propose a specific dataset placement strategy, and a joint uplink-downlink communication scheme exploiting coding at both the mobile users and the access point.

For a distributed computing application with K users each can store μ fractions of the dataset, the proposed CWDC scheme achieves the (normalized) communication loads

$$L_{\text{uplink}} \approx L_{\text{downlink}} \approx \frac{1}{\mu} - 1. \quad (1)$$

We note that the proposed scheme is scalable since the achieved communication loads are independent of K . As we show in Fig. 2, compared with a conventional uncoded shuffling scheme with a communication load $\mu K \cdot (\frac{1}{\mu} - 1)$ that explodes as the network expands, the proposed CWDC scheme reduces the load by a multiplicative factor of $\Theta(K)$.

We also extend our scalable design to a decentralized setting, in which the dataset placement is done at each user independently without knowing other collaborating users. For such a common scenario in mobile applications, we propose a decentralized scheme with a communication load close to that achieved in the centralized setting, particularly when the number of participating users is large.

Finally, we demonstrate that for both the centralized setting and the decentralized setting with a large number of users,

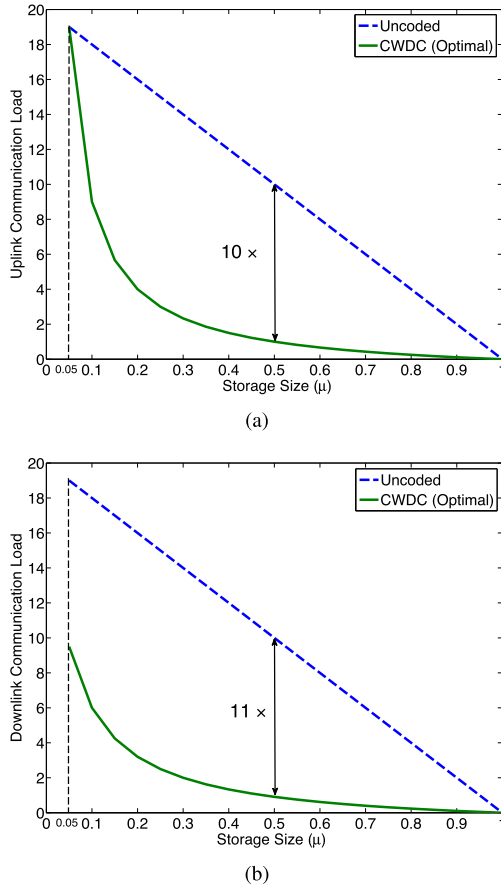


Fig. 2. Comparison of the communication loads achieved by the uncoded scheme with those achieved by the proposed CWDC scheme, for a network of $K = 20$ users. (a) Uplink. (b) Downlink.

the proposed CWDC schemes achieve the minimum possible communication loads that cannot be improved by any other scheme, by developing tight lower bounds.

B. Prior Works

The idea of applying coding in shuffling the intermediate results of MapReduce-like distributed computing frameworks was recently proposed in [8]–[11], for a wireline scenario where the computing nodes can directly communicate with each other through a shared link. In this paper, we consider a wireless distributed computing environment, in which wireless computing nodes communicate via an access point. More specifically, we extend the coded distributed computing framework in [8]–[11] in the following aspects.

- We extend the wireline setting in [9] and [10] to a wireless setting, and develop the first scalable framework with constant communication loads for wireless distributed computing.
- During data shuffling, other than designing codes at the users for uplink communication, we also design novel optimum code at the access point for downlink communication.
- We consider a decentralized setting that is of vital importance for wireless distributed computing, where each user has to decide its local storage content independently. We develop optimal decentralized dataset

placement strategy and uplink-downlink communication scheme to achieve the minimum communication loads asymptotically.

The idea of efficiently creating and exploiting coded multicasting opportunities was initially proposed in the context of cache networks in [14] and [15], and extended in [16] and [17], where caches pre-fetch part of the content in a way to enable coding during the content delivery, minimizing the network traffic. In this paper, we demonstrate that such coding opportunities can also be utilized to significantly reduce the communication load of wireless distributed computing applications. However, the proposed coded framework for wireless distributed computing differs significantly from the coded caching problems, mainly in the follow aspects.

- In [14] and [15], a central server has the entire dataset and broadcasts coded messages to satisfy users' demands. In this work, the access point neither stores any part of the dataset, nor performs any computation. We designed new codes at both the users and the access point for data shuffling.
- The cache contents are placed without knowing the users' demands in the coded caching problems, while here the dataset placement is performed knowing that each user has her own unique computation request (input).
- Our scheme is faced with the challenge of symmetric computation enforced by the MapReduce-type structure, i.e., a Map function computes intermediate values for all inputs. Such symmetry is not enforced in coded caching problems.

Other than reducing the communication load, a recent work [18] has also proposed to use codes to deal with the stragglers for a specific class of distributed computing jobs (e.g., matrix multiplication), and the optimal assignment of the coded tasks in a heterogeneous computing environment was addressed in [19]. Additionally, for these computing jobs, a unified coding framework was recently proposed in [20] to achieve a tradeoff between the latency of computation and the load of communication, on which the scheme in [10] and the scheme in [18] achieve the two end points, minimizing the communication load and the computation latency respectively.

There have also been several recent works on communication design and resource allocation for mobile-edge computation offloading (see e.g., [21], [22]), in which a part of the computation is offloaded to clouds located at the edges of cellular networks. In this scenario, recent works [23], [24] have proposed to exploit coding in edge processing to reduce the load of computation and improve the spectral efficiency. In contrast to the computation offloading model, in this paper, our focus is on the scenario that the "edge" only facilitates the communication required for distributed computing, and all computations are done *distributedly* at the users.

II. SYSTEM MODEL

We consider a system that has K mobile users, for some $K \in \mathbb{N}$. As illustrated in Fig. 3, all users are connected wirelessly to an access point (e.g., a cellular base station or a Wi-Fi router). The uplink channels of the K users towards the access point are orthogonal to each other, and the signals

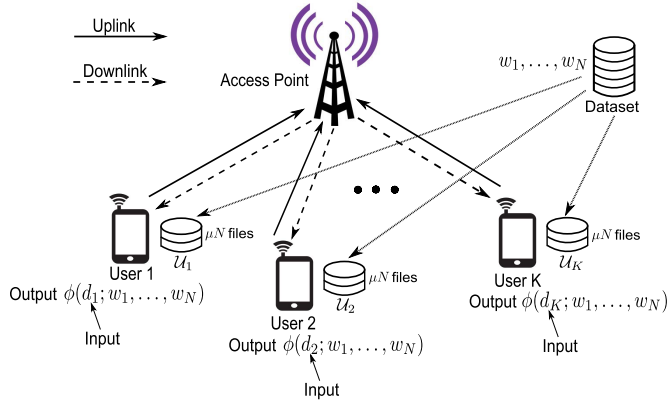


Fig. 3. A wireless distributed computing system.

transmitted by the access point on the downlink are received by all the users.

The system has a dataset (e.g., a feature repository of objects in a image recognition application) that is evenly partitioned into N files $w_1, \dots, w_N \in \mathbb{F}_{2^F}$, for some $N, F \in \mathbb{N}$. Every User k has a length- D input $d_k \in \mathbb{F}_{2^D}$ (e.g., user's image in the image recognition application) to process using the N files. To do that, as shown in Fig. 3, User k needs to compute

$$\phi\left(\underbrace{d_k}_{\text{input}}; \underbrace{w_1, \dots, w_N}_{\text{dataset}}\right), \quad (2)$$

where $\phi : \mathbb{F}_{2^D} \times (\mathbb{F}_{2^F})^N \rightarrow \mathbb{F}_{2^B}$ is an output function that maps the input d_k to an output result (e.g., the returned result after processing the image) of length $B \in \mathbb{N}$.

We assume that every mobile user has a local memory that can store up to μ fractions of the dataset (i.e., μN files), for some constant parameter μ that does not scale with the number of users K . Throughout the paper, we consider the case $\frac{1}{K} \leq \mu < 1$, such that each user does not have enough storage for the entire dataset, but the entire dataset can be stored collectively across all the users. We denote the set of indices of the files stored by User k as $\mathcal{U}_k$. The selections of $\mathcal{U}_k$ s are design parameters, and we denote the design of $\mathcal{U}_1, \dots, \mathcal{U}_K$ as *dataset placement*. The dataset placement is performed in prior to the computation (e.g., users download parts of the feature repository when installing the image recognition application).

Remark 1: The employed physical-layer network model is rather simple and one can do better using a more detailed model and more advanced techniques. However we note that any wireless medium can be converted to our simple model using (1) TDMA on uplink; and (2) broadcast at the rate of weakest user on downlink. Since the goal of the paper is to introduce a “coded” framework for scalable wireless distributed computing, we decide to abstract out the physical layer and focus on the amount of data needed to be communicated. $\square$

Distributed Computing Model: Motivated by prevalent distributed computing structures like MapReduce [12] and Spark [13], we assume that the computation for input d_k can be decomposed as

$$\phi(d_k; w_1, \dots, w_N) = h(g_1(d_k; w_1), \dots, g_N(d_k; w_N)), \quad (3)$$

where as illustrated in Fig. 4,

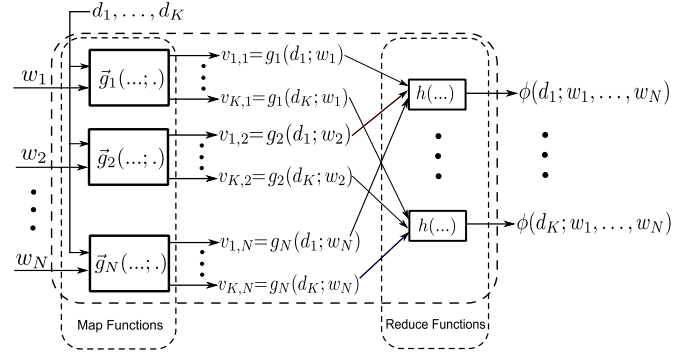


Fig. 4. A two-stage distributed computing framework decomposed into Map and Reduce functions.

- The “Map” functions $g_n(d_k; w_n) : \mathbb{F}_{2^D} \times \mathbb{F}_{2^F} \rightarrow \mathbb{F}_{2^T}$, $n \in \{1, \dots, N\}$, $k \in \{1, \dots, K\}$, maps the input d_k and the file w_n into an *intermediate value* $v_{k,n} = g_n(d_k; w_n) \in \mathbb{F}_{2^T}$, for some $T \in \mathbb{N}$,
- The “Reduce” function $h : (\mathbb{F}_{2^T})^N \rightarrow \mathbb{F}_{2^B}$ maps the intermediate values for input d_k in all files into the output value $\phi(d_k; w_1, \dots, w_N) = h(v_{k,1}, \dots, v_{k,N})$, for all $k \in \{1, \dots, K\}$.

Remark 2: Note that for every set of output functions such a Map-Reduce decomposition exists (e.g., setting g'_n s to identity and h to $\phi(d_k; *)$). However, such a decomposition is not unique, and in the distributed computing literature, there has been quite some work on developing appropriate decompositions of computations like join, sorting and matrix multiplication (see e.g., [12], [25]), which are suitable for efficient distributed computing. Here we do not impose any constraint on how the Map and Reduce functions are chosen (for example, they can be arbitrary *linear* or *non-linear* functions). $\square$

We focus on the applications in which the size of the users’ inputs is much smaller than the size of the computed intermediate values, i.e., $D \ll T$. As a result, the overhead of disseminating the inputs is negligible, and we assume that the users’ inputs $d_1, \dots, d_K$ are known at each user before the computation starts.

Remark 3: The above assumption holds for various wireless distributed computing applications. For example, in a mobile navigation application, an input is simply the addresses of the two end locations. The computed intermediate results contain all possible routes between the two end locations, from which the shortest one (or the fastest one considering the traffic condition) is computed for the user. Similarly, for a set of “filtering” applications like the aforementioned image recognition (or similarly augmented reality) and recommendation systems, the inputs are light-weight queries (e.g., the feature vector of an image) that are much smaller than the filtered intermediate results containing all attributes of related information. For example, an input can be multiple words describing the type of restaurant a user is interested in, and the intermediate results returned by a recommendation system application can be a list of relevant information that include customers’ comments, pictures, and videos of the recommended restaurants. $\square$

Following the decomposition in (3), the overall computation proceeds in three phases: *Map*, *Shuffle*, and *Reduce*.

Map Phase: User k , $k \in \{1, \dots, K\}$ computes the Map functions of $d_1, \dots, d_K$ based on the files in $\mathcal{U}_k$. For each input d_k and each file w_n in $\mathcal{U}_k$, User k computes $g_n(d_k, w_n) = v_{k,n}$.

Shuffle Phase: In order to compute the output value for the input d_k , User k needs the intermediate values that are *not* computed *locally* in the Map phase, i.e., $\{v_{k,n} : n \notin \mathcal{U}_k\}$. Users exchange the needed intermediate values via the access point they all wirelessly connect to. As a result, the Shuffle phase breaks into two sub-phases: *uplink communication* and *downlink communication*.

On the uplink, user k creates a message W_k as a function of the intermediate values computed locally, i.e., $W_k = \psi_k(\{v_{k,n} : k \in \{1, \dots, K\}, n \in \mathcal{U}_k\})$, and communicates W_k to the access point.

Definition 1 (Uplink Communication Load): We define the *uplink communication load*, denoted by L_u , as the total number of bits in all uplink messages $W_1, \dots, W_K$, normalized by the number of bits in the N intermediate values required by a user (i.e., NT).

We assume that the access point **does not have access to the dataset**. Upon decoding all the uplink messages $W_1, \dots, W_K$, the access point generates a message X from the decoded uplink messages, i.e., $X = \rho(W_1, \dots, W_K)$, then broadcasts X to all users on the downlink.

Definition 2 (Downlink Communication Load): We define the *downlink communication load*, denoted by L_d , as the number of bits in the downlink message X , normalized by NT .

Reduce Phase: User k , $k \in \{1, \dots, K\}$ uses the locally computed results $\{\tilde{g}_n : n \in \mathcal{U}_k\}$ and the decoded downlink message X to construct the inputs to the corresponding Reduce function, and calculates the output value $\phi(d_k; w_1, \dots, w_N) = h(v_{k,1}, \dots, v_{k,N})$.

Example (Uncoded Scheme): **As a benchmark**, we consider an uncoded scheme, where each user receives the needed intermediate values sent uncodedly by some other users and forwarded by the access point, achieving the communication loads $L_u^{\text{uncoded}}(\mu) = L_d^{\text{uncoded}}(\mu) = \mu K \cdot (\frac{1}{\mu} - 1)$.

We note that the above communication loads of the uncoded scheme grow with the number of users K , overwhelming the limited spectral resources. In this paper, we argue that by utilizing coding at the users and the access point, we can accommodate any number of users with a *constant* communication load. Particularly, we propose in the next section a scalable coded wireless distributed computing (CWDC) scheme that achieves the minimum possible uplink and downlink communication load simultaneously, i.e.,

$$L_u^{\text{coded}} = L_u^{\text{optimum}} \approx \frac{1}{\mu} - 1, \quad (4)$$

$$L_d^{\text{coded}} = L_d^{\text{optimum}} \approx \frac{1}{\mu} - 1. \quad (5)$$

III. THE PROPOSED CWDC SCHEME

In this section, we present the proposed CWDC scheme for a *centralized* setting, in which the dataset placement is

designed in a centralized manner knowing the number and the identities of the users that will participate in the computation. We first consider the storage size $\mu \in \{\frac{1}{K}, \frac{2}{K}, \dots, 1\}$ such that $\mu K \in \mathbb{N}$. **We assume that N is sufficiently large such that $N = \binom{K}{\mu K} \eta$ for some $\eta \in \mathbb{N}$.**

Dataset Placement and Map Phase Execution: We evenly partition the indices of the N files into $\binom{K}{\mu K}$ disjoint batches, each containing the indices of η files. We denote a batch of file indices as $\mathcal{B}_T$, which is labelled by a unique subset $T \subset \{1, \dots, K\}$ of size $|T| = \mu K$. As such defined, we have

$$\{1, \dots, N\} = \{i : i \in \mathcal{B}_T, T \subset \{1, \dots, K\}, |T| = \mu K\}. \quad (6)$$

User k , $k \in \{1, \dots, K\}$, stores locally all the files whose indices are in $\mathcal{B}_T$ if $k \in T$. That is,

$$\mathcal{U}_k = \bigcup_{T: |T| = \mu K, k \in T} \mathcal{B}_T. \quad (7)$$

As a result, each of the N files is stored by μK distinct users. After the Map phase, User k , $k \in \{1, \dots, K\}$, knows the intermediate values of all K output functions in each file whose index is in $\mathcal{U}_k$, i.e., $\{v_{q,n} : q \in \{1, \dots, K\}, n \in \mathcal{U}_k\}$.

In the example in Section I-A, the indices of the 6 files are partitioned into $\binom{3}{2} = 3$ batches, each containing the indices of 2 files. Each user stores the files whose indices are in 2 out of the 3 batches. Hence, each user stores a total of 4 files.

Uplink Communication: For any subset $\mathcal{W} \subset \{1, \dots, K\}$, and any $k \notin \mathcal{W}$, we denote the set of intermediate values needed by User k and known *exclusively* by users in $\mathcal{W}$ as $\mathcal{V}_{\mathcal{W}}^k$. More formally:

$$\mathcal{V}_{\mathcal{W}}^k \triangleq \{v_{k,n} : n \in \bigcap_{i \in \mathcal{W}} \mathcal{U}_i, n \notin \bigcup_{i \notin \mathcal{W}} \mathcal{U}_i\}. \quad (8)$$

In the example in Section I-A, we have $\mathcal{V}_{\{2,3\}}^1 = \{v_{1,5}, v_{1,6}\}$, $\mathcal{V}_{\{1,3\}}^2 = \{v_{2,1}, v_{2,2}\}$ and $\mathcal{V}_{\{1,2\}}^3 = \{v_{3,3}, v_{3,4}\}$.

For all subsets $\mathcal{S} \subset \{1, \dots, K\}$ of size $\mu K + 1$:

- 1) For each User $k \in \mathcal{S}$, $\mathcal{V}_{\mathcal{S} \setminus \{k\}}^k$ is the set of intermediate values that are requested by User k and are in the files whose indices are in the batch $\mathcal{B}_{\mathcal{S} \setminus \{k\}}$, and they are exclusively known at all users whose indices are in $\mathcal{S} \setminus \{k\}$. **We evenly and arbitrarily split $\mathcal{V}_{\mathcal{S} \setminus \{k\}}^k$, into μK disjoint segments $\{\mathcal{V}_{\mathcal{S} \setminus \{k\}, i}^k : i \in \mathcal{S} \setminus \{k\}\}$, where $\mathcal{V}_{\mathcal{S} \setminus \{k\}, i}^k$ denotes the segment associated with User i in $\mathcal{S} \setminus \{k\}$ for User k . That is, $\mathcal{V}_{\mathcal{S} \setminus \{k\}}^k = \bigcup_{i \in \mathcal{S} \setminus \{k\}} \mathcal{V}_{\mathcal{S} \setminus \{k\}, i}^k$.**
- 2) User i , $i \in \mathcal{S}$, sends the bit-wise XOR, denoted by $\oplus$, of all the segments associated with it in $\mathcal{S}$, i.e., User i sends the coded segment $W_i^{\mathcal{S}} \triangleq \bigoplus_{k \in \mathcal{S} \setminus \{i\}} \mathcal{V}_{\mathcal{S} \setminus \{k\}, i}^k$.

Since the coded message $W_i^{\mathcal{S}}$ contains $\frac{\eta}{\mu K} T^1$ bits for all $i \in \mathcal{S}$, there are a total of $\frac{(\mu K + 1)\eta}{\mu K} T$ bits communicated on the uplink in every subset $\mathcal{S}$ of size $\mu K + 1$. Therefore, the uplink communication load achieved by this coded scheme is $L_u^{\text{coded}}(\mu) = \frac{\binom{K}{\mu K + 1} (\mu K + 1) \cdot \eta \cdot T}{\mu K \cdot NT} = \frac{1}{\mu} - 1$, $\mu \in \{\frac{1}{K}, \frac{2}{K}, \dots, 1\}$.

Downlink Communication: For each subset $\mathcal{S} \subset \{1, \dots, K\}$ of size $\mu K + 1$, and $\mathcal{S} = \{i_1, i_2, \dots, i_{\mu K + 1}\}$,

¹Here we assume that T is sufficiently large such that $\frac{T}{\mu K} \in \mathbb{N}$.

the access point computes μK random linear combinations of the uplink messages generated based on the subset $\mathcal{S}$: $C_j^S(W_{i_1}^S, W_{i_2}^S, \dots, W_{i_{\mu K+1}}^S)$, $j = 1, \dots, \mu K$, and multicasts them to all users in $\mathcal{S}$.

Since each linear combination contains $\frac{\eta}{\mu K}T$ bits, the coded scheme achieves a downlink communication load $L_d^{\text{coded}}(\mu) = \frac{(\mu K + 1)\eta \cdot T}{NT} = \frac{\mu K}{\mu K + 1} \cdot (\frac{1}{\mu} - 1)$, $\mu \in \{\frac{1}{K}, \frac{2}{K}, \dots, 1\}$.

After receiving the random linear combinations $C_1^S, \dots, C_{\mu K}^S$, User i , $i \in \mathcal{S}$, cancels all segments she knows locally, i.e., $\bigcup_{k \in \mathcal{S} \setminus \{i\}} \{\mathcal{V}_{\mathcal{S} \setminus \{k\}, j}^k : j \in \mathcal{S} \setminus \{k\}\}$. Consequently, User i obtains μK random linear combinations of the required μK segments $\{\mathcal{V}_{\mathcal{S} \setminus \{i\}, j}^i : j \in \mathcal{S} \setminus \{i\}\}$.

Remark 4: The above uplink and downlink communication schemes require coding at both the users and the access point, creating multicasting messages that are simultaneously useful for many users. Such idea of efficiently creating and exploiting coded multicast opportunities was initially proposed in the coded caching problems in [14] and [15], and extended to D2D networks in [16]. While simply forwarding the coded uplink packets on the downlink can already reduce the downlink communication load by a factor of μK , performing random linear coding at the access point achieves a higher reduction factor of $\mu K + 1$. We note that this type of random linear coding at the access point has been utilized before in solving network coding problems (see, e.g., [26]–[28]) and bi-directional relaying problems (see, e.g., [29], [30]). $\square$

When μK is not an integer, we can first expand $\mu = \alpha\mu_1 + (1 - \alpha)\mu_2$ as a convex combination of $\mu_1 \triangleq \lfloor \mu K \rfloor / K$ and $\mu_2 \triangleq \lceil \mu K \rceil / K$. Then we partition the set of the N files into two disjoint subsets $\mathcal{I}_1$ and $\mathcal{I}_2$ of sizes $|\mathcal{I}_1| = \alpha N$ and $|\mathcal{I}_2| = (1 - \alpha)N$. We next apply the above coded scheme respectively to the files in $\mathcal{I}_1$ where each file is stored at $\mu_1 K$ users, and the files in $\mathcal{I}_2$ where each file is stored at $\mu_2 K$ users, yielding the following communication loads.

$$L_u^{\text{coded}}(\mu) = \alpha(\frac{1}{\mu_1} - 1) + (1 - \alpha)(\frac{1}{\mu_2} - 1), \quad (9)$$

$$L_d^{\text{coded}}(\mu) = \alpha \frac{\mu_1 K}{\mu_1 K + 1} \cdot (\frac{1}{\mu_1} - 1) + (1 - \alpha) \frac{\mu_2 K}{\mu_2 K + 1} \cdot (\frac{1}{\mu_2} - 1). \quad (10)$$

Hence, for general storage size μ , CWDC achieves the following communication loads.

$$L_u^{\text{coded}}(\mu) = \text{Conv}(\frac{1}{\mu} - 1), \quad (11)$$

$$L_d^{\text{coded}}(\mu) = \text{Conv}(\frac{\mu K}{\mu K + 1} \cdot (\frac{1}{\mu} - 1)), \quad (12)$$

where $\text{Conv}(f(\mu))$ denotes the lower convex envelope of the points $\{(\mu, f(\mu)) : \mu \in \{\frac{1}{K}, \frac{2}{K}, \dots, 1\}\}$.

We summarize the performance of the proposed CWDC scheme in the following theorem.

Theorem 1: For a wireless distributed computing application with a dataset of N files, and K users that each can store $\mu \in \{\frac{1}{K}, \frac{2}{K}, \dots, 1\}$ fraction of the files, the proposed CWDC scheme achieves the following uplink and downlink communication loads for sufficiently large N .

$$L_u^{\text{coded}}(\mu) = \frac{1}{\mu} - 1, \quad (13)$$

$$L_d^{\text{coded}}(\mu) = \frac{\mu K}{\mu K + 1} \cdot (\frac{1}{\mu} - 1). \quad (14)$$

For general $\frac{1}{K} \leq \mu \leq 1$, the achieved loads are as stated in (11) and (12).

Remark 5: Theorem 1 implies that, for large K , $L_u^{\text{coded}}(\mu) \approx L_d^{\text{coded}}(\mu) \approx \frac{1}{\mu} - 1$, which is independent of the number of users. Hence, we can accommodate any number of users without incurring extra communication load, and the proposed scheme is *scalable*. The reason for this phenomenon is that, as more users join the network, with an appropriate dataset placement, we can create coded multicasting opportunities to reduce the communication loads by a factor of μK , which is the size of the aggregated memory of all users in the system, and scales linearly with K (μ is a constant). Such phenomenon was also observed in the context of cache networks (see e.g., [14]). $\square$

Remark 6: As illustrated in Fig. 2 in Section I, compared with the uncoded scheme, the proposed CWDC scheme utilizes coding at the mobile users and the access point to reduce the uplink and downlink communication load by a factor of μK and $\mu K + 1$ respectively, which scale linearly with the number of users K . When $\mu = \frac{1}{K}$, which is the minimum storage size required to accomplish distributed computing, the CWDC scheme reduces to the uncoded scheme when the access point simply forwards the received uncoded packets. $\square$

Remark 7: Compared with distributed computing over wired servers where we only need to design one data shuffling scheme between servers in [10], here in the wireless setting we jointly design uplink and downlink shuffling schemes, which minimize both the uplink and downlink communication loads. $\square$

Remark 8: We can view the Shuffle phase as an instance of the index coding problem [31], [32], in which a central server aims to design a broadcast message with minimum length to satisfy the requests of all the clients, given the clients' local side information. While a random linear network coding approach (see e.g., [26]–[28]) is sufficient to implement any multicast communication, it is generally sub-optimal for index coding problems where every client requests different messages. However, for the considered wireless distributed computing scenario where we are given the flexibility of designing dataset placement (thus the side information), we can prove that the proposed CWDC scheme is optimum in minimizing communication loads (see Section V). $\square$

Remark 9: We note that the coding opportunities created and exploited in the proposed coded scheme belong to a type of in-network coding, which aims to combat interference in wireless networks, and deliver the information bits required by each of the users respectively with maximum spectral efficiency. This type of coding is distinct from source coding, or data compression (see e.g., [33]), which aims to remove the redundant information in the original intermediate values each of the users requests. Interestingly, the above proposed coded communication scheme can be applied on top of data compression. That is, we can first compress the intermediate values to minimize the number of information bits each user requests, then we apply the proposed coded communication scheme on the compressed values, in order to deliver them

to intended users with minimum utilization of the wireless links. $\square$

So far, we have considered the scenario where the dataset placement is designed in a centralized manner, i.e., the dataset placement is designed knowing which users will use the application. However, a more practical scenario is that before computation, the dataset placement at each user is performed in a decentralized manner without knowing when the computation will take place and who will take part in the computation. In the next section, we describe how we can extend the proposed CWDC scheme to facilitate the computation in such a decentralized setting.

IV. THE PROPOSED CWDC SCHEME FOR THE DECENTRALIZED SETTING

We consider a decentralized system, in which a random and a priori unknown subset of users, denoted by $\mathcal{K}$, participate in the computation. The dataset placement is performed *independently* at each user by *randomly* storing a subset of μN files, according to a common placement distribution P . In this case, we define the *information loss* of the system, denoted by Δ , as the fraction of the files that are not stored by any participating user.

Once the computation starts, the participating users in $\mathcal{K}$ of size K are fixed, and their identities are revealed to all the participating users. Then they collaboratively perform the computation as in the centralized setting. The participating users process their inputs over the available part of the dataset stored collectively by all participating users. More specifically, every user k in $\mathcal{K}$ now computes

$$\phi\left(\underbrace{d_k}_{\text{input}}; \underbrace{\{w_n : n \in \bigcup_{k \in \mathcal{K}} \mathcal{U}_k\}}_{\text{available dataset}}\right). \quad (15)$$

In what follows, we present the proposed CWDC scheme for the above decentralized setting, including a random dataset placement strategy, an uplink communication scheme and a downlink communication scheme.

Dataset Placement: We use a *uniformly random* dataset placement, in which every user independently stores μN files uniformly at random. With high probability for large N , the information loss approximately equals $(1 - \mu)^K$, which converges quickly to 0 as K increases.

For a decentralized random dataset placement, files are stored by random subsets of users. During data shuffling, we first *greedily* categorize the available files based on the number of users that store the file, then for each category we deliver the corresponding intermediate values in an *opportunistic* way using the coded communication schemes described in Section III for the centralized setting.

Uplink Communication: For all subsets $\mathcal{S} \subseteq \{1, \dots, K\}$ with size $|\mathcal{S}| \geq 2$:

- 1) For each $k \in \mathcal{S}$, we evenly and arbitrarily split $\mathcal{V}_{\mathcal{S} \setminus \{k\}}^k$ defined in (8), into $|\mathcal{S}| - 1$ disjoint segments $\mathcal{V}_{\mathcal{S} \setminus \{k\}}^k = \{\mathcal{V}_{\mathcal{S} \setminus \{k\}, i}^k : i \in \mathcal{S} \setminus \{k\}\}$, and associate the segment $\mathcal{V}_{\mathcal{S} \setminus \{k\}, i}^k$ with the user $i \in \mathcal{S} \setminus \{k\}$.

- 2) User $i, i \in \mathcal{S}$, sends the bit-wise XOR, denoted by $\oplus$, of all the segments associated with it in $\mathcal{S}$, i.e., User i sends the coded segment $W_i^{\mathcal{S}} \triangleq \bigoplus_{k \in \mathcal{S} \setminus \{i\}} \mathcal{V}_{\mathcal{S} \setminus \{k\}, i}^k$.²

Using the proposed uniformly random dataset placement, for any subset $\mathcal{S} \subseteq \{1, \dots, K\}$, the number of files exclusively stored by all users in $\mathcal{S}$ can be characterized by $\mu^{|\mathcal{S}|}(1 - \mu)^{K - |\mathcal{S}|}N + o(N)$.

Thus, when the proposed communication scheme proceeds on a subset $\mathcal{S}$ of size $|\mathcal{S}| = j + 1$ users, the resulting uplink communication load converges to $\frac{j+1}{j}\mu^j(1 - \mu)^{K-j}$ for large N .

Downlink Communication: For all $\mathcal{S} \subseteq \{1, \dots, K\}$ of size $|\mathcal{S}| \geq 2$, the access point computes $|\mathcal{S}| - 1$ random linear combinations of the uplink messages generated based on the subset $\mathcal{S}$: $C_j^{\mathcal{S}}(\{W_i^{\mathcal{S}} : i \in \mathcal{S}\})$, $j = 1, \dots, |\mathcal{S}| - 1$, and multicasts them to all users in $\mathcal{S}$.

We summarize the performance of the proposed decentralized CWDC scheme in the following theorem.

Theorem 2: *For an application with a dataset of N files, and K users that each can store μ fraction of the files, the proposed decentralized CWDC scheme achieves an information loss $\Delta = (1 - \mu)^K$ and the following communication loads with high probability for sufficiently large N .*

$$L_{\text{decent}, u}^{\text{coded}} = \sum_{j=1}^{K-1} \binom{K}{j+1} \frac{j+1}{j} \mu^j (1 - \mu)^{K-j}, \quad (16)$$

$$L_{\text{decent}, d}^{\text{coded}} = \sum_{j=1}^{K-1} \binom{K}{j+1} \mu^j (1 - \mu)^{K-j}. \quad (17)$$

Remark 10: In Fig. 5, we numerically evaluate the communication loads achieved by the proposed centralized and decentralized schemes, in a network with 20 participating users. We observe that although the loads of the decentralized scheme are higher than those of the centralized scheme, the communication performances under these two settings are very close to each other. As K becomes large, the information loss achieved by the decentralized CWDC approaches 0, and both loads in (17) approach $\frac{1}{\mu} - 1$, which equals the asymptotic loads achieved by the centralized scheme (see Remark 5). Hence, when the number of participating users is large, there is little loss in making the system decentralized. $\square$

Remark 11: To understand the fact that the proposed decentralized scheme performs close to the centralized one when the number of participating users is large, we notice the fact that when uniformly random dataset placement is used, as demonstrated in Fig. 6, almost all files are stored by approximately μK users for large K , which coincides with the optimal dataset placement for the centralized setting. Thus, coding gains of the the proposed decentralized communication schemes would be also very close to those of the centralized schemes. Such phenomenon was also observed in [15] for caching problems with decentralized content placement. $\square$

²Since the dataset placement is now randomized, we zero-pad all elements in $\{\mathcal{V}_{\mathcal{S} \setminus \{k\}, i}^k : k \in \mathcal{S} \setminus \{i\}\}$ to the maximum length $\max_{k \in \mathcal{S} \setminus \{i\}} |\mathcal{V}_{\mathcal{S} \setminus \{k\}, i}^k|$ in order to complete the XOR operation.

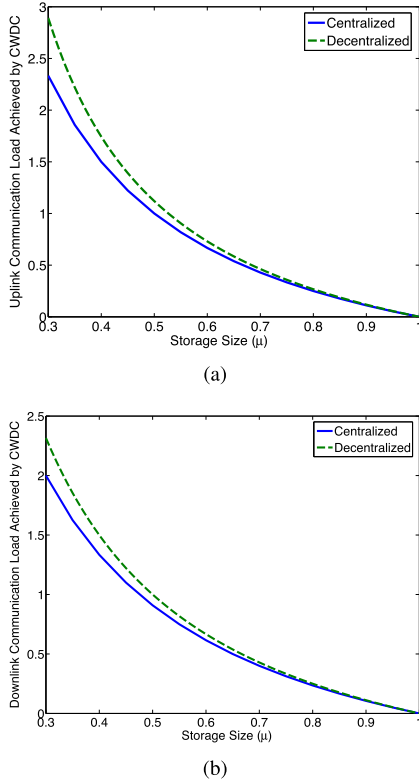


Fig. 5. Comparison of the communication loads achieved by the centralized and the decentralized CWDC schemes, for a network of $K = 20$ participating users. (a) Uplink. (b) Downlink.

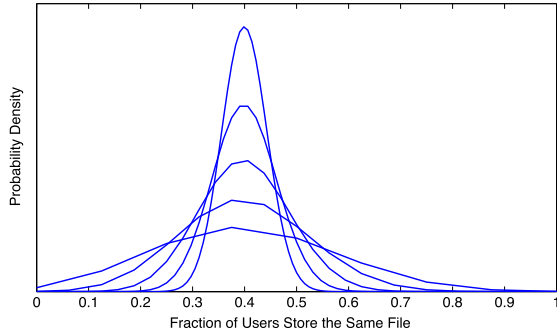


Fig. 6. Concentration of the number of users each files is stored at around μK . Each curve demonstrates the normalized fraction of files that are stored by different numbers of users, for a particular number of participating users K . The density functions are computed for a storage size $\mu = 0.4$, and for $K = 2^3, \dots, 2^7$.

V. OPTIMALITY OF THE PROPOSED CWDC SCHEMES

In this section, we demonstrate in the following two theorems, that the proposed CWDC schemes achieve the minimum uplink and downlink communication loads using any scheme, for the centralized setting and the decentralized setting respectively.

Theorem 3: *For a centralized wireless distributed computing application using any dataset placement and communication schemes that achieve an uplink load L_u and a downlink load L_d , L_u and L_d are lower bounded by $L_u^{\text{coded}}(\mu)$ and $L_d^{\text{coded}}(\mu)$ as stated in Theorem 1 respectively.*

Remark 12: Using Theorem 1 and 3, we have completely characterized the minimum achievable uplink and downlink communication loads, using *any* dataset placement, uplink and downlink communication schemes for the centralized setting. This implies that the proposed centralized CWDC scheme *simultaneously* minimizes both uplink and downlink communication loads required to accomplish distributed computing, and no other scheme can improve upon it. This also demonstrates that there is no fundamental tension between optimizing uplink and downlink communication in wireless distributed computing. $\square$

For a dataset placement $\mathcal{U} = \{\mathcal{U}_k\}_{k=1}^K$, we denote the minimum possible uplink and downlink communication loads, achieved by any uplink-downlink communication scheme to accomplish wireless distributed computing, by $L_u^*(\mathcal{U})$ and $L_d^*(\mathcal{U})$ respectively. We next prove Theorem 3 by deriving lower bounds on $L_u^*(\mathcal{U})$ and $L_d^*(\mathcal{U})$ respectively.

A. Lower Bound on $L_u^*(\mathcal{U})$

For a given dataset placement $\mathcal{U}$, we denote the number of files that are stored at j users as $a_{\mathcal{U}}^j$, for all $j \in \{1, \dots, K\}$, i.e.,

$$a_{\mathcal{U}}^j = \sum_{\mathcal{J} \subseteq \{1, \dots, K\}: |\mathcal{J}|=j} |(\bigcap_{k \in \mathcal{J}} \mathcal{U}_k) \setminus (\bigcup_{i \notin \mathcal{J}} \mathcal{U}_i)|. \quad (18)$$

For any $\mathcal{U}$, it is clear that $\{a_{\mathcal{U}}^j\}_{j=1}^K$ satisfy

$$\sum_{j=1}^K a_{\mathcal{U}}^j = N, \quad (19)$$

$$\sum_{j=1}^K j a_{\mathcal{U}}^j = \mu N K. \quad (20)$$

We start the proof with the following lemma, which characterizes a lower bound on $L_u^*(\mathcal{U})$ in terms of the distribution of the files in the dataset placement $\mathcal{U}$, i.e., $a_{\mathcal{U}}^1, \dots, a_{\mathcal{U}}^K$.

$$\text{Lemma 1: } L_u^*(\mathcal{U}) \geq \sum_{j=1}^K \frac{a_{\mathcal{U}}^j}{N} \cdot \frac{K-j}{j}.$$

Proof Sketch: We know from the model of the distributed computing system that given the local computation results, and the downlink broadcast message X , each mobile user should be able to recover all the required intermediate values for the local Reduce function. Since the downlink message X is generated as a function of the uplink messages $W_1, \dots, W_K$ at the access point, a user can of course recover the required intermediate values if she were given $W_1, \dots, W_K$ instead of X . Having observed the above fact, we can then prove Lemma 1 following the similar steps in the proof of Lemma 1 in [10], in which each user can broadcast her message, generated as a function of the local computation results, to all other users. $\blacksquare$

Lemma 1 implies that in order to deliver an intermediate value of size T bits that is known at j users and needed by one of the remaining $K - j$ users, the j users who know this value need to communicate at least T/j bits on the uplink.

Next, since the function $\frac{K-j}{j}$ in Lemma 1 is convex in j , and by (19) that $\sum_{j=1}^K \frac{a_{\mathcal{U}}^j}{N} = 1$ and (20), we have

$$L_u^*(\mathcal{U}) \geq \frac{K - \sum_{j=1}^K j \frac{a_{\mathcal{U}}^j}{N}}{\sum_{j=1}^K j \frac{a_{\mathcal{U}}^j}{N}} = \frac{K - \mu K}{\mu K} = \frac{1}{\mu} - 1. \quad (21)$$

$\mu = \frac{M}{N}$
 $r = \mu K$
 $\frac{K}{\mu}$

We can further improve the lower bound in (21) for a particular μ such that $\mu K \notin \mathbb{N}$. For a given storage size μ , we first find two points $(\mu_1, \frac{1}{\mu_1} - 1)$ and $(\mu_2, \frac{1}{\mu_2} - 1)$, where $\mu_1 \triangleq \lfloor \mu K \rfloor / K$ and $\mu_2 \triangleq \lceil \mu K \rceil / K$. Then we find the line $p + qt$ connecting these two points as a function of t , $\frac{1}{K} \leq t \leq 1$, for some constants $p, q \in \mathbb{R}$. We note that p and q are different for different μ and

$$p + qt|_{t=\mu_1} = \frac{1}{\mu_1} - 1, \quad (22)$$

$$p + qt|_{t=\mu_2} = \frac{1}{\mu_2} - 1. \quad (23)$$

Then by the convexity of the function $\frac{1}{t} - 1$, the function $\frac{1}{t} - 1$ cannot be smaller than the function $p + qt$ at the points $t = \frac{1}{K}, \frac{2}{K}, \dots, 1$. That is, for all $t \in \{\frac{1}{K}, \dots, 1\}$,

$$\frac{1}{t} - 1 \geq p + qt. \quad (24)$$

By Lemma 1, we have

$$L_u^*(\mathcal{U}) \geq \sum_{j=1}^K \frac{a_{\mathcal{U}}^j}{N} \cdot \frac{K-j}{j} \quad (25)$$

$$= \sum_{t=\frac{1}{K}, \dots, 1} \frac{a_{\mathcal{U}}^{tK}}{N} \cdot \left(\frac{1}{t} - 1\right) \quad (26)$$

$$\geq \sum_{t=\frac{1}{K}, \dots, 1} \frac{a_{\mathcal{U}}^{tK}}{N} \cdot (p + qt) \quad (27)$$

$$= p + q\mu, \quad (28)$$

Therefore, for general $\frac{1}{K} \leq \mu \leq 1$, $L_u^*(\mathcal{U})$ is lower bounded by the lower convex envelope of the points $\{(\mu, \frac{1}{\mu} - 1) : \mu \in \{\frac{1}{K}, \frac{2}{K}, \dots, 1\}\}$.

B. Lower Bound on $L_d^*(\mathcal{U})$

The lower bound on the minimum downlink communication load $L_d^*(\mathcal{U})$ can be proved following the similar steps of lower bounding the minimum uplink communication load $L_u^*(\mathcal{U})$, after making the following enhancements to the downlink communication system:

- We consider the access point as the $(K+1)$ th user who has stored all N files and has a virtual input to process. Thus the enhanced downlink communication system has $K+1$ users, and the dataset placement for the enhanced system

$$\bar{\mathcal{U}} \triangleq \{\mathcal{U}, \mathcal{U}_{K+1}\}, \quad (29)$$

where $\mathcal{U}_{K+1}$ is equal to $\{1, \dots, N\}$.

- We assume that every one of the $K+1$ users can broadcast to the rest of the users, where the broadcast message is generated by mapping the locally stored files.

Apparently the minimum downlink communication load of the system cannot increase after the above enhancements. Thus the lower bound on the minimum downlink communication load of the enhanced system is also a lower bound for the original system.

Then we can apply the same arguments in the proof of Lemma 1 to the enhanced downlink system of $K+1$ users, obtaining a lower bound on $L_d^*(\mathcal{U})$, as described in the following corollary:

Corollary 1: $L_d^*(\mathcal{U}) \geq \sum_{j=1}^K \frac{a_{\mathcal{U}}^j}{N} \cdot \frac{K-j}{j+1}$.

Proof: Applying Lemma 1 to the enhanced downlink system yields

$$L_d^*(\bar{\mathcal{U}}) \geq \sum_{j=1}^{K+1} \frac{a_{\bar{\mathcal{U}}}^j}{N} \cdot \frac{K+1-j}{j} \geq \sum_{j=2}^{K+1} \frac{a_{\bar{\mathcal{U}}}^j}{N} \cdot \frac{K+1-j}{j} \quad (30)$$

$$= \sum_{j=1}^K \frac{a_{\mathcal{U}}^{j+1}}{N} \cdot \frac{K-j}{j+1}. \quad (31)$$

Since the access point has stored every file, $a_{\bar{\mathcal{U}}}^{j+1} = a_{\mathcal{U}}^j$, for all $j \in \{1, \dots, K\}$. Therefore, (31) can be re-written as

$$L_d^*(\mathcal{U}) \geq L_d^*(\bar{\mathcal{U}}) \geq \sum_{j=1}^K \frac{a_{\mathcal{U}}^j}{N} \cdot \frac{K-j}{j+1}. \quad (32)$$

Then following the same arguments as in the proof for the minimum uplink communication load, we have

$$L_d^*(\mathcal{U}) \geq \frac{K-\mu K}{\mu K+1} = \frac{\mu K}{\mu K+1} \cdot \left(\frac{1}{\mu} - 1\right). \quad (33)$$

For general $\frac{1}{K} \leq \mu \leq 1$, $L_d^*(\mathcal{U})$ is lower bounded by the lower convex envelope of the points $\{(\mu, \frac{\mu K}{\mu K+1} (\frac{1}{\mu} - 1)) : \mu \in \{\frac{1}{K}, \frac{2}{K}, \dots, 1\}\}$.

This completes the proof of Theorem 3.

Theorem 4: Consider a decentralized wireless distributed computing application. For any random dataset placement with a placement distribution P that achieves an information loss Δ , and communication schemes that achieve communication loads L_u and L_d with high probability for large N , L_u and L_d are lower bounded by $\frac{1}{\mu} - 1$ when K is large and Δ approaches 0.

Remark 13: When the number of participating users is large (large K), the above lower bound in Theorem 4 coincides with the asymptotic loads achieved by the proposed decentralized CWDC scheme stated in Theorem 2 (see Remark 10). Therefore, the proposed decentralized scheme is asymptotically optimal. $\square$

We now prove Theorem 4 by showing that for any decentralized dataset placement, the minimum achievable communication loads are lower bounded by $\frac{1}{\mu} - 1$ when the number of participating users is large and the information loss approaches zero. Hence, the asymptotic communication loads achieved by the proposed decentralized scheme can not be further improved. In particular, for a particular realization of the dataset placement $\mathcal{U}$ with information loss $\Delta(\mathcal{U})$, we denote the minimum possible uplink and downlink communication loads by $L_{\text{decent},u}^*(\mathcal{U})$ and $L_{\text{decent},d}^*(\mathcal{U})$, and derive lower bounds on $L_{\text{decent},u}^*(\mathcal{U})$ and $L_{\text{decent},d}^*(\mathcal{U})$ respectively.

We note that given the information loss $\Delta(\mathcal{U})$, $1 - \Delta(\mathcal{U})$ fraction of files are available across the participating users, all of which need to be processed to compute the outputs (see (15)). Among those files, $\bar{\mu}(\mathcal{U}) \triangleq \frac{\mu}{1-\Delta(\mathcal{U})}$ fraction of them are stored by each participating user. Following the same steps in proving the lower bounds of the centralized setting, the minimum communication loads for the dataset placement $\mathcal{U}$ are lower bounded as follows.

$$L_{\text{decent},u}^*(\mathcal{U}) \geq \left(\frac{1}{\bar{\mu}(\mathcal{U})} - 1 \right) (1 - \Delta(\mathcal{U})) \quad (34)$$

$$= \left(\frac{1 - \Delta(\mathcal{U})}{\mu} - 1 \right) (1 - \Delta(\mathcal{U})), \quad (35)$$

$$L_{\text{decent},d}^*(\mathcal{U}) \geq \frac{\bar{\mu}(\mathcal{U})K}{\bar{\mu}(\mathcal{U})K + 1} \left(\frac{1}{\bar{\mu}(\mathcal{U})} - 1 \right) (1 - \Delta(\mathcal{U})) \quad (36)$$

$$= \frac{\mu K}{\mu K + 1 - \Delta(\mathcal{U})} \left(\frac{1 - \Delta(\mathcal{U})}{\mu} - 1 \right) \times (1 - \Delta(\mathcal{U})). \quad (37)$$

Since the above bounds hold for any realization of dataset placement $\mathcal{U}$, for a decentralized dataset placement scheme with a distribution P that achieves an information loss $\Delta(P)$, communication loads $L_{\text{decent},u}^*(P)$, $L_{\text{decent},d}^*(P)$ with high probability, the following inequalities hold.

$$L_{\text{decent},u}^*(P) \geq \left(\frac{1 - \Delta(P)}{\mu} - 1 \right) (1 - \Delta(P)), \quad (38)$$

$$L_{\text{decent},d}^*(P) \geq \frac{\mu K}{\mu K + 1 - \Delta(P)} \left(\frac{1 - \Delta(P)}{\mu} - 1 \right) \times (1 - \Delta(P)). \quad (39)$$

Hence, when the number of active users are large, the achievable uplink and downlink communication loads, for any decentralized dataset placement scheme with a distribution P that achieves a vanishing information loss are bounded by

$$L_{\text{decent},u}^*(P) \geq \frac{1}{\mu} - 1, \quad (40)$$

$$L_{\text{decent},d}^*(P) \geq \frac{1}{\mu} - 1. \quad (41)$$

This completes the proof of Theorem 4.

VI. CONCLUSIONS AND FUTURE DIRECTIONS

In this paper, we proposed a scalable wireless distributed computing framework, for both the centralized and the decentralized settings, such that the shuffling load does not increase with the number of participating users. In particular, we use a repetitive placement of the dataset across the users to enable coding, reducing the shuffling load by a factor that scales linearly with the network size.

In this paper, we have abstracted out several practical challenges to demonstrate the theoretical feasibility of a scalable wireless distributed computing framework and quantify the resulting gains. Future directions can be to generalize the model to also incorporate the following important aspects.

- *Network Heterogeneity.* Most mobile networks are heterogeneous. Different mobile devices have different link quality, processing power, battery capacity, and QoS

requirement. For example, the proposed coded computing schemes are for a set of mobile users with similar uplink/downlink channel strengths and communication rates. When users have heterogeneous link capacities, one straightforward solution is to first partition the users into groups, such that all users within a group have similar channel strength, and then apply the proposed schemes within each group. However, designing the optimal grouping strategy is a challenging problem that requires further exploration. Other than performing the computations at the users themselves, the superior computation and storage capacity of a growing number of edge servers at access points encourage computation offloading to the edge servers. Another interesting problem is to consider a mobile network of heterogeneous users with the possibility of performing computations at the edge servers, and study the optimal scheduling and communication schemes (see e.g., [21]).

- *Computation Heterogeneity.* In many wireless distributed computing applications (especially for graph processing), the intermediate computation results have heterogeneous sizes. For example, for a navigation application over a highly clustered map, some parts of the map generate much more useful information than the other parts, resulting in highly skewed intermediate results. In such scenario, the proposed coding scheme still applies, but the coding operations are not symmetric as in the case of homogeneous intermediate results (e.g., one may now need to compute the XOR of two data segments with different sizes). Alternatively, we can consider a low-complexity greedy approach, in which we perform the dataset placement to maximize the number of multicasting opportunities that simultaneously deliver useful information to the largest possible number of users. Nevertheless, finding the optimal dataset placement and coding scheme in the case of heterogeneous computation results remains a challenging open problem.
- *Straggling/Failing Users.* So far we have assumed that for both the centralized and the decentralized settings, once the collaborative computation process starts, all participating users are active and reliable until the end of the computation. However, similar to the straggler problems in wireline computer clusters (see e.g., [34]), one needs to account for the possibilities of mobile users losing connectivity, leaving, and joining the application in the middle of computation. One approach to deal with straggling/failing users during the computation process is to assign users coded computation tasks using e.g., Maximum-Distance-Separable codes (see [18] for an example of applying coded computations on matrix multiplication). Using this approach, the successful execution of the mobile application can be achieved by retrieving the computation results from only a subset of “healthy” users, and this can provide the system with certain level of robustness to straggling/failing users during the course of computation.
- *Multi-Stage Computation.* We have so far designed the schemes for applications with one stage of

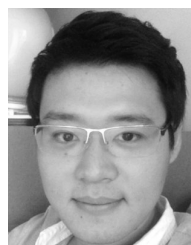
Map-Reduce computation. However, a general application contains multiple stages of computations, interconnected as a directed acyclic graph (see e.g., [35]). It would be interesting to understand the optimal schemes for such general applications. A preliminary exploration along this direction was recently presented in [36].

ACKNOWLEDGMENT

The views, opinions, and/or findings expressed are those of the author(s) and should not be interpreted as representing the official views or policies of the Department of Defense or the U.S. Government.

REFERENCES

- [1] S. Li, Q. Yu, M. A. Maddah-Ali, and A. S. Avestimehr, "Poster abstract: A scalable coded computing framework for edge-facilitated wireless distributed computing," in *Proc. IEEE/ACM Symp. Edge Comput. (SEC)*, Oct. 2016, pp. 79–80.
- [2] S. Li, Q. Yu, M. A. Maddah-Ali, and A. S. Avestimehr, "Edge-facilitated wireless distributed computing," in *Proc. IEEE GLOBECOM*, Dec. 2016, pp. 1–7.
- [3] F. Bonomi, R. Milito, J. Zhu, and S. Addepalli, "Fog computing and its role in the Internet of Things," in *Proc. 1st Ed. MCC Workshop Mobile Cloud Comput.*, 2012, pp. 13–16.
- [4] U. Drolia *et al.*, "The case for mobile edge-clouds," in *Proc. IEEE 10th Int. Conf. Ubiquitous Intell. Comput. (UIC)*, Dec. 2013, pp. 209–215.
- [5] D. Datla *et al.*, "Wireless distributed computing: A survey of research challenges," *IEEE Commun. Mag.*, vol. 50, no. 1, pp. 144–152, Jan. 2012.
- [6] G. Huerta-Canepa and D. Lee, "A virtual cloud computing provider for mobile devices," in *Proc. 1st ACM Workshop Mobile Cloud Comput. Services Soc. Netw. Beyond*, 2010, Art. no. 1.
- [7] M. Chowdhury, M. Zaharia, J. Ma, M. I. Jordan, and I. Stoica, "Managing data transfers in computer clusters with orchestra," *ACM SIGCOMM Comput. Commun. Rev.*, vol. 41, no. 4, pp. 98–109, Aug. 2011.
- [8] S. Li, M. A. Maddah-Ali, and A. S. Avestimehr, "Coded MapReduce," in *Proc. 53rd Allerton Conf.*, Sep. 2015, pp. 964–971.
- [9] S. Li, M. A. Maddah-Ali, and A. S. Avestimehr, "Fundamental tradeoff between computation and communication in distributed computing," in *Proc. IEEE ISIT*, Jul. 2016, pp. 1814–1818.
- [10] S. Li, M. A. Maddah-Ali, Q. Yu, and A. S. Avestimehr, "A fundamental tradeoff between computation and communication in distributed computing," [Online]. Available: <https://arxiv.org/abs/1604.07086>
- [11] S. Li, S. Supittayapornpong, M. A. Maddah-Ali, and A. S. Avestimehr, "Coded terasort," in *Proc. 6th Int. Workshop Parallel Distrib. Comput. Large Scale Mach. Learn. Big Data Anal.*, to be published.
- [12] J. Dean and S. Ghemawat, "MapReduce: Simplified data processing on large clusters," in *Proc. 6th USENIX (OSDI)*, Dec. 2004, p. 10.
- [13] M. Zaharia, M. Chowdhury, M. J. Franklin, S. Shenker, and I. Stoica, "Spark: Cluster computing with working sets," in *Proc. 2nd USENIX HotCloud*, Jun. 2010, p. 10.
- [14] M. A. Maddah-Ali and U. Niesen, "Fundamental limits of caching," *IEEE Trans. Inf. Theory*, vol. 60, no. 5, pp. 2856–2867, May 2014.
- [15] M. A. Maddah-Ali and U. Niesen, "Decentralized coded caching attains order-optimal memory-rate tradeoff," *IEEE/ACM Trans. Netw.*, vol. 23, no. 4, pp. 1029–1040, Aug. 2015.
- [16] M. Ji, G. Caire, and A. F. Molisch, "Fundamental limits of caching in wireless D2D networks," *IEEE Trans. Inf. Theory*, vol. 62, no. 2, pp. 849–869, Feb. 2016.
- [17] N. Karamchandani, U. Niesen, M. A. Maddah-Ali, and S. Diggavi, "Hierarchical coded caching," in *Proc. IEEE ISIT*, Jul. 2014, pp. 2142–2146.
- [18] K. Lee, M. Lam, R. Pedarsani, D. Papailiopoulos, and K. Ramchandran, "Speeding up distributed machine learning using codes," in *Proc. NIPS, Workshop Mach. Learn. Syst.*, Dec. 2015, pp. 1143–1147.
- [19] A. Reiszadehmobarakeh, S. Prakash, R. Pedarsani, and S. Avestimehr, "Coded computation over heterogeneous clusters," in *Proc. IEEE ISIT*, to be published.
- [20] S. Li, M. A. Maddah-Ali, and A. S. Avestimehr, "A unified coding framework for distributed computing with straggling servers," in *Proc. IEEE NetCod*, Dec. 2016, pp. 1–6.
- [21] S. Barbarossa, S. Sardellitti, and P. D. Lorenzo, "Communicating while computing: Distributed mobile cloud computing over 5G heterogeneous networks," *IEEE Signal Process. Mag.*, vol. 31, no. 6, pp. 45–55, Nov. 2014.
- [22] S. Khalili and O. Simeone, "Inter-layer per-mobile optimization of cloud mobile computing: A message-passing approach," *Trans. Emerg. Telecommun. Technol.*, vol. 27, no. 6, pp. 814–827, 2016.
- [23] S. Li, M. A. Maddah-Ali, and A. S. Avestimehr, "Coding for distributed fog computing," *IEEE Commun. Mag.*, vol. 55, no. 4, pp. 34–40, Apr. 2017.
- [24] S. Li, M. A. Maddah-Ali, and A. S. Avestimehr, "Communication-aware computing for edge processing," in *Proc. IEEE ISIT*, to be published.
- [25] A. Rajaraman and J. D. Ullman, *Mining of Massive Datasets*. Cambridge, U.K.: Cambridge Univ. Press, 2011.
- [26] R. Ahlswede, N. Cai, S.-Y. R. Li, and R. W. Yeung, "Network information flow," *IEEE Trans. Inf. Theory*, vol. 46, no. 4, pp. 1204–1216, Jul. 2000.
- [27] R. Koetter and M. Médard, "An algebraic approach to network coding," *IEEE/ACM Trans. Netw.*, vol. 11, no. 5, pp. 782–795, Oct. 2003.
- [28] T. Ho, R. Koetter, M. Médard, D. R. Karger, and M. Effros, "The benefits of coding over routing in a randomized setting," in *Proc. IEEE ISIT*, Jun. 2003, p. 442.
- [29] S. J. Kim, P. Mitran, and V. Tarokh, "Performance bounds for bidirectional coded cooperation protocols," *IEEE Trans. Inf. Theory*, vol. 54, no. 11, pp. 5235–5241, Nov. 2008.
- [30] A. S. Avestimehr, A. Sezgin, and D. N. C. Tse, "Capacity of the two-way relay channel within a constant gap," *Eur. Trans. Telecommun.*, vol. 21, no. 4, pp. 363–374, 2010.
- [31] Y. Birk and T. Kol, "Coding on demand by an informed source (ISCOD) for efficient broadcast of different supplemental data to caching clients," *IEEE Trans. Inf. Theory*, vol. 52, no. 6, pp. 2825–2830, Jun. 2006.
- [32] Z. Bar-Yossef, Y. Birk, T. S. Jayram, and T. Kol, "Index coding with side information," *IEEE Trans. Inf. Theory*, vol. 57, no. 3, pp. 1479–1494, Mar. 2011.
- [33] B. Welton *et al.*, "Improving I/O forwarding throughput with data compression," in *Proc. IEEE Int. Conf. Cluster Comput.*, Jul. 2011, pp. 438–445.
- [34] M. Zaharia, A. Konwinski, A. D. Joseph, R. H. Katz, and I. Stoica, "Improving MapReduce performance in heterogeneous environments," *Proc. OSDI*, vol. 8, no. 4, p. 7, Dec. 2008.
- [35] B. Saha *et al.*, "Apache Tez: A unifying framework for modeling and building data processing applications," in *Proc. ACM SIGMOD Int. Conf. Manage. Data*, May 2015, pp. 1357–1369.
- [36] S. Li, M. A. Maddah-Ali, and A. S. Avestimehr, "Coded distributed computing: Straggling servers and multistage dataflows," in *Proc. 54th Allerton Conf.*, Sep. 2016, pp. 164–171.



Songze Li (S'09) received the B.S. degree in electrical engineering from the Polytechnic Institute of New York University (currently NYU Tandon School of Engineering) in 2011, and the M.S. degree in electrical engineering from the University of Southern California in 2016, where he is currently pursuing the Ph.D. degree. He is a Research Assistant with the Electrical Engineering Department, University of Southern California. His research interest is network information theory and its applications including improving parallel/distributed computing using codes, and interference management in wireless networks.

He is a Qualcomm Innovation Fellowship finalist in 2017, and received the USC Viterbi School of Engineering Doctoral Fellowship in 2011.



Qian Yu received the B.S. degree in electrical engineering and computer science and physics and the M.Eng. degree in electrical engineering from the Massachusetts Institute of Technology. He is currently pursuing the Ph.D. degree in electrical engineering with the Viterbi School of Engineering, University of Southern California. His interests span information theory, distributed computing, and many other problems math-related. He is a Qualcomm Innovation Fellowship finalist in 2017, and received the Annenberg Graduate Fellowship in 2015. He received Honorable Mention in the William Lowell Putnam Mathematical Competition in 2013.



Mohammad Ali Maddah-Ali (S'03–M'08) received the B.Sc. degree in electrical engineering from the Isfahan University of Technology, and the M.A.Sc. degree in electrical engineering from the University of Tehran. He is currently pursuing the Ph.D. degree with the University of Waterloo, Canada. From 2002 to 2007, he was with the Coding and Signal Transmission Laboratory, Department of Electrical and Computer Engineering, University of Waterloo. From 2007 to 2008, he was with the Wireless Technology Laboratories, Nortel Networks, Ottawa, ON, Canada. From 2008 to 2010, he was a Post-Doctoral Fellow with the Department of Electrical Engineering and Computer Sciences, University of California at Berkeley. Then, he joined Bell Labs, Holmdel, NJ, USA, as a Communication Research Scientist. He is currently with the Sharif University of Technology as a Faculty Member.

He was a recipient of the NSERC Post-Doctoral Fellowship in 2007, the Best Paper Award at the IEEE International Conference on Communications in 2014, the IEEE Communications Society and the IEEE Information Theory Society Joint Paper Award in 2015, and the IEEE Information Theory Society Joint Paper Award in 2016.



A. Salman Avestimehr (S'04–M'09–SM'17) received the B.S. degree in electrical engineering from the Sharif University of Technology in 2003, the M.S. and Ph.D. degrees in electrical engineering and computer science from the University of California at Berkeley, Berkeley, CA, USA, in 2005 and 2008, respectively. He is currently an Associate Professor with the Department of Electrical Engineering, University of Southern California. His current research interests include information theory, the theory of communications, and their applications to distributed computing and data analytics.

He has received a number of awards, including the IEEE Communications Society and Information Theory Society Joint Paper Award, the Presidential Early Career Award for Scientists and Engineers (PECASE) for Pushing the Frontiers of Information Theory Through Its Extension to Complex Wireless Information Networks, the Young Investigator Program Award from the U.S. Air Force Office of Scientific Research, the National Science Foundation CAREER Award, and the David J. Sakrison Memorial Prize. He is currently an Associate Editor of the IEEE TRANSACTIONS ON INFORMATION THEORY.